

Submission Worksheet

Submission Data

Course: IT202-007-F2025

Assignment: IT202 Milestone 2

Student: Dylan G. (dg599)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 802.00/802.00 (100.00%)

Received Grade: 0.00/802.00 (0.00%)

Started: 11/24/2025 6:42:19 PM

Updated: 11/24/2025 11:14:33 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-2/grading/dg599>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-2/view/dg599>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0

1. Refer to Milestone2 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>

2. Ensure you read all instructions and objectives before starting.

3. Ensure you've gone through each lesson related to this Milestone

4. Switch to the Milestone2 branch

1. `git checkout Milestone2` (ensure proper starting branch)
2. `git pull origin Milestone2` (ensure history is up to date)

5. Fill out the below worksheet

- Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
- Ensure proper styling is applied to each page
- Ensure there are no visible technical errors; only user-friendly messages are allowed

6. Once finished, click "Submit and Export"

7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github

1. `git add .`
2. `git commit -m "adding PDF"`
3. `git push origin Milestone2`
4. On Github merge the pull request from Milestone2 to dev
5. On Github create a pull request from dev to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)

8. Upload the same PDF to Canvas

9. Sync Local

10. `git checkout dev`

11. `git pull origin dev`

Section #1: (2 pts.) Data

Progress: 100%

≡ Task #1 (0 50 pts) - API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond `id`, `created`, `modified`, and respective `is_api` or `api_id` columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an `api_id` or `is_api` column)

Q	id	api_id	is_api	make	model	year	type	image url	description	user	created timestamp	modified timestamp
1	(NULL)	0		Ford	Mustang	2020	Sedan	https://www.ford.com/cars/		5	2025-11-24 15:43:29	2025-11-24 15:43:29
3	subaru-ecu-4-24-271hp-1	1		Subaru	WRX (VR) II	2025	Sedan	https://www.subaru.com/	2.4 (271 Hp) AND 5MT Sed	5	2025-11-24 19:53:40	2025-11-24 19:53:40
4	subaru-ecu-4-24-271hp-1	1		Subaru	WRX (VR) II	2025	Sedan	https://www.subaru.com/	2.4 (271 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
5	subaru-ecu-4-24-271hp-1	1		Subaru	WRX Sedan (VA) (1648 cc)	2025	Sedan	https://www.subaru.com/	2.0 (268 Hp) AND CVT Sed	5	2025-11-24 19:53:40	2025-11-24 19:53:40
6	subaru-ecu-4-24-271hp-1	1		Subaru	WRX Sedan (VA) (1648 cc)	2025	Sedan	https://www.subaru.com/	2.0 (268 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
7	subaru-ecu-4-24-271hp-1	1		Subaru	WRX STI (1648 cc)	2025	Sedan	https://www.subaru.com/	2.0 (268 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
8	subaru-ecu-4-24-271hp-1	1		Subaru	WRX STI (1648 cc)	2025	Sedan	https://www.subaru.com/	2.0 (268 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
9	subaru-ecu-4-24-271hp-1	1		Subaru	WRX STI (1648 cc)	2025	Sedan	https://www.subaru.com/	2.5 (311 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
10	subaru-ecu-4-24-271hp-1	1		Subaru	WRX STI (1648 cc)	2025	Sedan	https://www.subaru.com/	2.5 (311 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
11	subaru-ecu-4-24-271hp-1	1		Subaru	WRX Sedan (VA)	2025	Sedan	https://www.subaru.com/	2.0 (268 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40
12	subaru-ecu-4-24-271hp-1	1		Subaru	WRX STI	2025	Sedan	https://www.subaru.com/	2.5 (311 Hp) AND Sedan, i	5	2025-11-24 19:53:40	2025-11-24 19:53:40

database table

```

CREATE TABLE IF NOT EXISTS cars (
  id VARCHAR(255) PRIMARY KEY,
  api_id VARCHAR(255),
  is_api BOOLEAN,
  make VARCHAR(255),
  model VARCHAR(255),
  year INT,
  type VARCHAR(255),
  image_url VARCHAR(255),
  description VARCHAR(255),
  user VARCHAR(255),
  created_timestamp DATETIME,
  modified_timestamp DATETIME
);

INSERT INTO cars (id, api_id, is_api, make, model, year, type, image_url, description, user, created_timestamp, modified_timestamp)
VALUES ('subaru-ecu-4-24-271hp-1', '1', '1', 'Subaru', 'WRX (VR) II', '2025', 'Sedan', 'https://www.subaru.com/', '2.4 (271 Hp) AND 5MT | Sed', '5', '2025-11-24 19:53:40', '2025-11-24 19:53:40');

```

sql file



Saved: 11/24/2025 7:52:02 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in `.sql`)

URL #1

<https://github.com/dylangarca/dg599-IT20240741>


URL

<https://github.com/dylangarca/dg>



Saved: 11/24/2025 7:52:02 PM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

The id field is the primary key and is a unique identifier for each car in the database. API id is a unique id that is provided by the car api and this field is null when the car is created manually. is api field is showign whether the car is added through the api with a 1 or if the car was created manually with a 0. make is car's brand, model is the specific model of the car, year is the production of the year, type is category of the car, image url is where the url link of the image of the car, description is a short description of the car. user id is the id of the user who added the car into the database. created and modified are timestamps of when the car was created or modified. idx make,year, and api id are indexes to make the queiries quicker.



Saved: 11/24/2025 7:52:02 PM

Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the data transformation/conversion (from the API)
- Show a Heroku dev example of API getting fetched and loaded into the DB

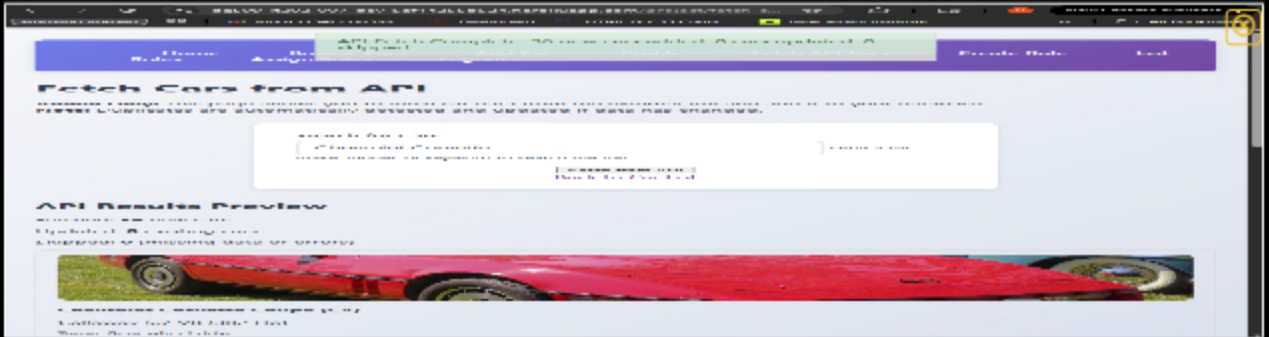
```
1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

process api code p1

```
1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

```
1 #!/usr/bin/perl
2 use strict;
3 use warnings;
4 use LWP::Simple;
5 use JSON;
6 use Data::Dumper;
7
8 my $url = "https://api.carfax.net/v1/cars/";
9 my $token = "YOUR_API_TOKEN";
10
11 # Fetch cars from the API
12 my $response = get($url . "?token=$token");
13 my $json = decode_json($response);
14
15 # Print the results
16 print Dumper($json);
```

process api code p2



dev example of fetch

 Saved: 11/24/2025 8:02:15 PM

Part 2:

Progress: 100%

Details:

- Include Heroku prod link to API fetch page
- Include pull request link for this feature

URL #1

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/fetch_cars.php



URL

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/fetch_cars.php



URL #2

<https://github.com/dylangarca/dg599-it202-007-dev-5af14255b524>



URL

<https://github.com/dylangarca/dg599-it202-007-dev-5af14255b524>



 Saved: 11/24/2025 8:02:15 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

I am using a subset of the data, id is mapping to api id, the title gets split into make and model, content is used for the description of the car, additional is used to determine the car type, image is for the image url. I extract the make and model of the cars from the title string and then I convert the additional field has phrases and I take certain parts of that field to get the a clean type of the car. Then I combine the content and additional field to make the description. So the goal is get the messy api data and turn it into clean values like when we split the title into the make and model field, detect the car type from additional by using keyword matching, and also sanitizing inputs.



Saved: 11/24/2025 8:02:15 PM

⇒ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

So based off the two given cases, in the first case of the entity exist but the data wasn't modified what will happen is that Update will execute but all the values are going to stay the same because there is no change, all this case will do is increase the updated count. The user will see that no new cars were added, and shows X cars updated and Y cars skipped based on the number of cars. Now in the second case of the entity exist but the api data has been modified it will execute an update and update the fields that got changed and it will display a similar string to the user like the previous case.



Saved: 11/24/2025 8:07:44 PM

⇒ Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)
 - What happens if you get the same result from the API for an entity that



php validation



⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic

- Details:**

 - Briefly explain the html validations used
 - Briefly explain the javascript validation logic
 - Briefly explain the php validation logic

Your Response:

For the html form the key validation attributes that were used are required and i used this attribute for make,model, and year, this make sure that the user can't submit the form without these important fields. The next attribute is maxlength these are used to limit the amount of characters in the make, model, and type field. The next attribute is type number and this is used so that the user can not input characters into the year field. another type that is used is type url and i use this to make sure that the format for the url is correct. Then the last attribute for html is min 1900 and max current date + 1 and this is ensuring that the user enters a valid year range. For the js validation we have validate(form) and this is ensuring that before submitting the form the valjues of make, model, and year are valid inputs. So for make it checks if there are at least 2 characters, for model it is checking that is not empty, and then also making sure that the year is in the correct range. And if any of those three fail it will flash a warning message. For the php form the script is getting the values from the sanitize function, it is checking that make, model is not empty and year is not empty and it is the correct range. if any of the validations fail it will give a flash message, if the validation passes then it will create the car and add in the cars table.



Task #2 (60 pts.) - Examples

Progress: 100%

— Task Collapsed —

Task #3 (30 pts.) - Links Progress: 100%

Progress: 100%

Details:

- Include the heroku prod link to this page
- Include pull request link for this feature

URL #1

https://dg599-it202-007-prod-0134f160fb38.herokuapp.com/project/create_car.php



URL

<https://dg599-it202-007-prod>



URL #2

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



URL

<https://github.com/dylangarc>



Saved: 11/24/2025 7:14:42 PM

Section #3: (150 pts.) Data List Page (Many Items)

Progress: 100%

≡ Task #1 (60 pts.) - Code

Progress: 100%

📁 Part 1:

Progress: 100%

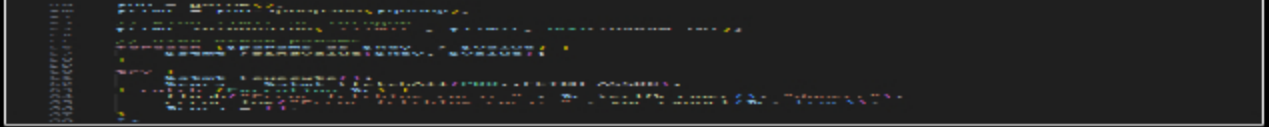
Details:

- Show the code that handles the filter/sort logic
- Show the code that generates the list/grid output
- Styling must be applied

```
1  <!-- Filter and Sort Logic -->
2  <div class="filter-sort">
3    <div>
4      <input type="text" value="Filter by name" />
5      <input type="button" value="Filter" />
6    </div>
7    <div>
8      <input type="radio" checked="" value="asc" /> Ascending
9      <input type="radio" value="desc" /> Descending
10     <input type="button" value="Sort" />
11   </div>
12 </div>
13
14 <div class="list">
15   <table>
16     <thead>
17       <tr>
18         <th>ID</th>
19         <th>Name</th>
20         <th>Price</th>
21       </tr>
22     </thead>
23     <tbody>
24       <!-- Loop through cars -->
25       <@php foreach($cars as $car) {
26         <tr>
27           <td>{$car['id']}</td>
28           <td>{$car['name']}</td>
29           <td>{$car['price']}</td>
30         </tr>
31       }</tbody>
32     </table>
33   </div>
```

filter/sort logic p1

```
1  <!-- List/Grid Output -->
2  <div class="list">
3    <table>
4      <thead>
5        <tr>
6          <th>ID</th>
7          <th>Name</th>
8          <th>Price</th>
9        </tr>
10      </thead>
11      <tbody>
12        <!-- Loop through cars -->
13        <@php foreach($cars as $car) {
14          <tr>
15            <td>{$car['id']}</td>
16            <td>{$car['name']}</td>
17            <td>{$car['price']}</td>
18          </tr>
19        }</tbody>
20      </table>
21    </div>
```

filter/sort logic p2



output code



Saved: 11/24/2025 8:47:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the filter/sort logic works and formulates the results
- Briefly explain how the list/grid output is generated
- Briefly note the styling choices

Your Response:

For the logic the code is building a dynamic SQL query based on the filters that get filled out by the user, then the code will validate all the inputs, it also will bind the parameters for the query securely, and lastly the code fetches the filtered/sorted list of cars. For the output, after the SQL query is run it is checking the cars array to see if it is empty and if so it will display a message of no results available but if it contains data then it will loop through the array then for each car a card is created and it shows an image of a car, the make, model, year, type, and wheter if the car was created by the api or manually. For the styling i use mainly bootstrap with light custom styling for a clean card look, spacing a readability.



Saved: 11/24/2025 8:47:06 PM

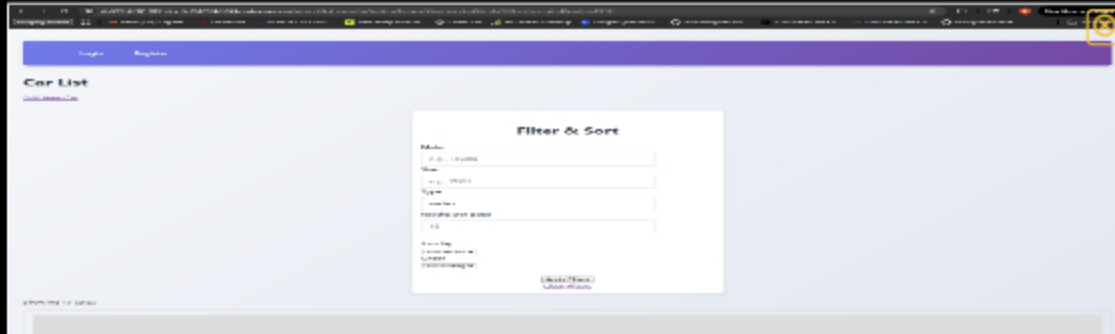
Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records)
- Each list item should be a summary

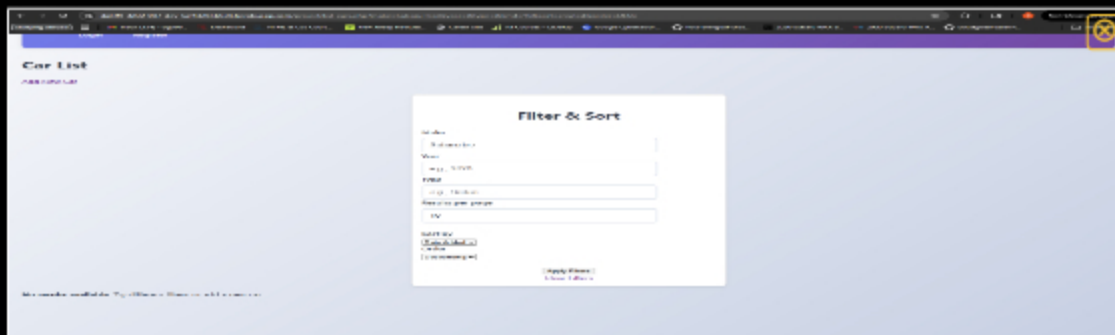
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure heroku url is visible



sedan sort p1



sedan sort p2



no results



Saved: 11/24/2025 8:53:52 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the heroku prod link to this page
- Include pull request link for this feature

URL #1

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/list_cars.php

URL

<https://dg599-it202-007-dev-!>



URL #2

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



URL

<https://github.com/dylangarc>



Saved: 11/24/2025 8:54:44 PM

Section #4: (150 pts.) View Details Page (Single Item)

Progress: 100%

≡ Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the entity output
- Styling must be applied

```
// dig.php
// dig.php 13/04/2017 - "http://partialis.net/nav.php":
require_once($_SERVER["DOCUMENT_ROOT"]."/partials/nav.php");

$car_id = (int)$e($_GET["id"], 0, false);

if ($car_id <= 0) {
    flash("Invalid car ID", "danger");
    die(header("Location: index.php"));
}

$car = getDBC();
$stmt = $car->prepare("SELECT * FROM cars WHERE id = :id LIMIT 1");

try {
    $stmt->execute([":id" => $car_id]);
    $car = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$car) {
        flash("Car not found", "danger");
        die(header("Location: index.php"));
    }
} catch (PDOException $e) {
    flash("Error connecting to database", "danger");
    var_dump($e->getMessage());
    die(header("Location: index.php"));
}
}
```

code that handles loading of the record

```

// 1. 定义一个名为 Person 的类，包含姓名、年龄、性别属性
class Person {
public:
    string name;
    int age;
    bool sex;
};

// 2. 定义一个名为 Student 的类，继承自 Person，并添加学号属性
class Student : public Person {
public:
    int id;
};

// 3. 定义一个名为 Teacher 的类，继承自 Person，并添加职称属性
class Teacher : public Person {
public:
    string title;
};

// 4. 定义一个名为 Employee 的类，继承自 Person，并添加工资属性
class Employee : public Person {
public:
    double salary;
};

// 5. 定义一个名为 Manager 的类，继承自 Employee，并添加管理权限属性
class Manager : public Employee {
public:
    bool manage;
};

// 6. 定义一个名为 Doctor 的类，继承自 Teacher，并添加医生属性
class Doctor : public Teacher {
public:
    bool doctor;
};

// 7. 定义一个名为 Nurse 的类，继承自 Doctor，并添加护士属性
class Nurse : public Doctor {
public:
    bool nurse;
};

// 8. 定义一个名为 Engineer 的类，继承自 Employee，并添加工程师属性
class Engineer : public Employee {
public:
    bool engineer;
};

// 9. 定义一个名为 Scientist 的类，继承自 Engineer，并添加科学家属性
class Scientist : public Engineer {
public:
    bool scientist;
};

// 10. 定义一个名为 Programmer 的类，继承自 Scientist，并添加程序员属性
class Programmer : public Scientist {
public:
    bool programmer;
};

// 11. 定义一个名为 Researcher 的类，继承自 Programmer，并添加研究员属性
class Researcher : public Programmer {
public:
    bool researcher;
};

// 12. 定义一个名为 Analyst 的类，继承自 Researcher，并添加分析师属性
class Analyst : public Researcher {
public:
    bool analyst;
};

// 13. 定义一个名为 Designer 的类，继承自 Analyst，并添加设计师属性
class Designer : public Analyst {
public:
    bool designer;
};

// 14. 定义一个名为 Developer 的类，继承自 Designer，并添加开发人员属性
class Developer : public Designer {
public:
    bool developer;
};

// 15. 定义一个名为 Tester 的类，继承自 Developer，并添加测试人员属性
class Tester : public Developer {
public:
    bool tester;
};

// 16. 定义一个名为 QA 的类，继承自 Tester，并添加质量保证属性
class QA : public Tester {
public:
    bool qa;
};

// 17. 定义一个名为 Product Manager 的类，继承自 Manager，并添加产品经理属性
class ProductManager : public Manager {
public:
    bool productManager;
};

// 18. 定义一个名为 Project Manager 的类，继承自 Manager，并添加项目经理属性
class ProjectManager : public Manager {
public:
    bool projectManager;
};

// 19. 定义一个名为 Team Lead 的类，继承自 Manager，并添加团队负责人属性
class TeamLead : public Manager {
public:
    bool teamLead;
};

// 20. 定义一个名为 Senior Engineer 的类，继承自 Engineer，并添加高级工程师属性
class SeniorEngineer : public Engineer {
public:
    bool seniorEngineer;
};

// 21. 定义一个名为 Junior Engineer 的类，继承自 Engineer，并添加初级工程师属性
class JuniorEngineer : public Engineer {
public:
    bool juniorEngineer;
};

// 22. 定义一个名为 Senior Scientist 的类，继承自 Scientist，并添加高级科学家属性
class SeniorScientist : public Scientist {
public:
    bool seniorScientist;
};

// 23. 定义一个名为 Junior Scientist 的类，继承自 Scientist，并添加初级科学家属性
class JuniorScientist : public Scientist {
public:
    bool juniorScientist;
};

// 24. 定义一个名为 Senior Programmer 的类，继承自 Programmer，并添加高级程序员属性
class SeniorProgrammer : public Programmer {
public:
    bool seniorProgrammer;
};

// 25. 定义一个名为 Junior Programmer 的类，继承自 Programmer，并添加初级程序员属性
class JuniorProgrammer : public Programmer {
public:
    bool juniorProgrammer;
};

// 26. 定义一个名为 Senior Researcher 的类，继承自 Researcher，并添加高级研究员属性
class SeniorResearcher : public Researcher {
public:
    bool seniorResearcher;
};

// 27. 定义一个名为 Junior Researcher 的类，继承自 Researcher，并添加初级研究员属性
class JuniorResearcher : public Researcher {
public:
    bool juniorResearcher;
};

// 28. 定义一个名为 Senior Analyst 的类，继承自 Analyst，并添加高级分析师属性
class SeniorAnalyst : public Analyst {
public:
    bool seniorAnalyst;
};

// 29. 定义一个名为 Junior Analyst 的类，继承自 Analyst，并添加初级分析师属性
class JuniorAnalyst : public Analyst {
public:
    bool juniorAnalyst;
};

// 30. 定义一个名为 Senior Designer 的类，继承自 Designer，并添加高级设计师属性
class SeniorDesigner : public Designer {
public:
    bool seniorDesigner;
};

// 31. 定义一个名为 Junior Designer 的类，继承自 Designer，并添加初级设计师属性
class JuniorDesigner : public Designer {
public:
    bool juniorDesigner;
};

// 32. 定义一个名为 Senior Developer 的类，继承自 Developer，并添加高级开发人员属性
class SeniorDeveloper : public Developer {
public:
    bool seniorDeveloper;
};

// 33. 定义一个名为 Junior Developer 的类，继承自 Developer，并添加初级开发人员属性
class JuniorDeveloper : public Developer {
public:
    bool juniorDeveloper;
};

// 34. 定义一个名为 Senior Tester 的类，继承自 Tester，并添加高级测试人员属性
class SeniorTester : public Tester {
public:
    bool seniorTester;
};

// 35. 定义一个名为 Junior Tester 的类，继承自 Tester，并添加初级测试人员属性
class JuniorTester : public Tester {
public:
    bool juniorTester;
};

// 36. 定义一个名为 Senior QA 的类，继承自 QA，并添加高级质量保证属性
class SeniorQA : public QA {
public:
    bool seniorQA;
};

// 37. 定义一个名为 Junior QA 的类，继承自 QA，并添加初级质量保证属性
class JuniorQA : public QA {
public:
    bool juniorQA;
};

// 38. 定义一个名为 Senior Product Manager 的类，继承自 ProductManager，并添加高级产品经理属性
class SeniorProductManager : public ProductManager {
public:
    bool seniorProductManager;
};

// 39. 定义一个名为 Junior Product Manager 的类，继承自 ProductManager，并添加初级产品经理属性
class JuniorProductManager : public ProductManager {
public:
    bool juniorProductManager;
};

// 40. 定义一个名为 Senior Project Manager 的类，继承自 ProjectManager，并添加高级项目经理属性
class SeniorProjectManager : public ProjectManager {
public:
    bool seniorProjectManager;
};

// 41. 定义一个名为 Junior Project Manager 的类，继承自 ProjectManager，并添加初级项目经理属性
class JuniorProjectManager : public ProjectManager {
public:
    bool juniorProjectManager;
};

// 42. 定义一个名为 Senior Team Lead 的类，继承自 TeamLead，并添加高级团队负责人属性
class SeniorTeamLead : public TeamLead {
public:
    bool seniorTeamLead;
};

// 43. 定义一个名为 Junior Team Lead 的类，继承自 TeamLead，并添加初级团队负责人属性
class JuniorTeamLead : public TeamLead {
public:
    bool juniorTeamLead;
};

// 44. 定义一个名为 Senior Senior Engineer 的类，继承自 SeniorEngineer，并添加高级高级工程师属性
class SeniorSeniorEngineer : public SeniorEngineer {
public:
    bool seniorSeniorEngineer;
};

// 45. 定义一个名为 Junior Senior Engineer 的类，继承自 SeniorEngineer，并添加初级高级工程师属性
class JuniorSeniorEngineer : public SeniorEngineer {
public:
    bool juniorSeniorEngineer;
};

// 46. 定义一个名为 Senior Junior Engineer 的类，继承自 JuniorEngineer，并添加高级初级工程师属性
class SeniorJuniorEngineer : public JuniorEngineer {
public:
    bool seniorJuniorEngineer;
};

// 47. 定义一个名为 Junior Junior Engineer 的类，继承自 JuniorEngineer，并添加初级初级工程师属性
class JuniorJuniorEngineer : public JuniorEngineer {
public:
    bool juniorJuniorEngineer;
};

// 48. 定义一个名为 Senior Senior Scientist 的类，继承自 SeniorScientist，并添加高级高级科学家属性
class SeniorSeniorScientist : public SeniorScientist {
public:
    bool seniorSeniorScientist;
};

// 49. 定义一个名为 Junior Senior Scientist 的类，继承自 SeniorScientist，并添加初级高级科学家属性
class JuniorSeniorScientist : public SeniorScientist {
public:
    bool juniorSeniorScientist;
};

// 50. 定义一个名为 Senior Junior Scientist 的类，继承自 JuniorScientist，并添加高级初级科学家属性
class SeniorJuniorScientist : public JuniorScientist {
public:
    bool seniorJuniorScientist;
};

// 51. 定义一个名为 Junior Junior Scientist 的类，继承自 JuniorScientist，并添加初级初级科学家属性
class JuniorJuniorScientist : public JuniorScientist {
public:
    bool juniorJuniorScientist;
};

// 52. 定义一个名为 Senior Senior Programmer 的类，继承自 SeniorProgrammer，并添加高级高级程序员属性
class SeniorSeniorProgrammer : public SeniorProgrammer {
public:
    bool seniorSeniorProgrammer;
};

// 53. 定义一个名为 Junior Senior Programmer 的类，继承自 SeniorProgrammer，并添加初级高级程序员属性
class JuniorSeniorProgrammer : public SeniorProgrammer {
public:
    bool juniorSeniorProgrammer;
};

// 54. 定义一个名为 Senior Junior Programmer 的类，继承自 JuniorProgrammer，并添加高级初级程序员属性
class SeniorJuniorProgrammer : public JuniorProgrammer {
public:
    bool seniorJuniorProgrammer;
};

// 55. 定义一个名为 Junior Junior Programmer 的类，继承自 JuniorProgrammer，并添加初级初级程序员属性
class JuniorJuniorProgrammer : public JuniorProgrammer {
public:
    bool juniorJuniorProgrammer;
};

// 56. 定义一个名为 Senior Senior Researcher 的类，继承自 SeniorResearcher，并添加高级高级研究员属性
class SeniorSeniorResearcher : public SeniorResearcher {
public:
    bool seniorSeniorResearcher;
};

// 57. 定义一个名为 Junior Senior Researcher 的类，继承自 SeniorResearcher，并添加初级高级研究员属性
class JuniorSeniorResearcher : public SeniorResearcher {
public:
    bool juniorSeniorResearcher;
};

// 58. 定义一个名为 Senior Junior Researcher 的类，继承自 JuniorResearcher，并添加高级初级研究员属性
class SeniorJuniorResearcher : public JuniorResearcher {
public:
    bool seniorJuniorResearcher;
};

// 59. 定义一个名为 Junior Junior Researcher 的类，继承自 JuniorResearcher，并添加初级初级研究员属性
class JuniorJuniorResearcher : public JuniorResearcher {
public:
    bool juniorJuniorResearcher;
};

// 60. 定义一个名为 Senior Senior Analyst 的类，继承自 SeniorAnalyst，并添加高级高级分析师属性
class SeniorSeniorAnalyst : public SeniorAnalyst {
public:
    bool seniorSeniorAnalyst;
};

// 61. 定义一个名为 Junior Senior Analyst 的类，继承自 SeniorAnalyst，并添加初级高级分析师属性
class JuniorSeniorAnalyst : public SeniorAnalyst {
public:
    bool juniorSeniorAnalyst;
};

// 62. 定义一个名为 Senior Junior Analyst 的类，继承自 JuniorAnalyst，并添加高级初级分析师属性
class SeniorJuniorAnalyst : public JuniorAnalyst {
public:
    bool seniorJuniorAnalyst;
};

// 63. 定义一个名为 Junior Junior Analyst 的类，继承自 JuniorAnalyst，并添加初级初级分析师属性
class JuniorJuniorAnalyst : public JuniorAnalyst {
public:
    bool juniorJuniorAnalyst;
};

// 64. 定义一个名为 Senior Senior Designer 的类，继承自 SeniorDesigner，并添加高级高级设计师属性
class SeniorSeniorDesigner : public SeniorDesigner {
public:
    bool seniorSeniorDesigner;
};

// 65. 定义一个名为 Junior Senior Designer 的类，继承自 SeniorDesigner，并添加初级高级设计师属性
class JuniorSeniorDesigner : public SeniorDesigner {
public:
    bool juniorSeniorDesigner;
};

// 66. 定义一个名为 Senior Junior Designer 的类，继承自 JuniorDesigner，并添加高级初级设计师属性
class SeniorJuniorDesigner : public JuniorDesigner {
public:
    bool seniorJuniorDesigner;
};

// 67. 定义一个名为 Junior Junior Designer 的类，继承自 JuniorDesigner，并添加初级初级设计师属性
class JuniorJuniorDesigner : public JuniorDesigner {
public:
    bool juniorJuniorDesigner;
};

// 68. 定义一个名为 Senior Senior Developer 的类，继承自 SeniorDeveloper，并添加高级高级开发人员属性
class SeniorSeniorDeveloper : public SeniorDeveloper {
public:
    bool seniorSeniorDeveloper;
};

// 69. 定义一个名为 Junior Senior Developer 的类，继承自 SeniorDeveloper，并添加初级高级开发人员属性
class JuniorSeniorDeveloper : public SeniorDeveloper {
public:
    bool juniorSeniorDeveloper;
};

// 70. 定义一个名为 Senior Junior Developer 的类，继承自 JuniorDeveloper，并添加高级初级开发人员属性
class SeniorJuniorDeveloper : public JuniorDeveloper {
public:
    bool seniorJuniorDeveloper;
};

// 71. 定义一个名为 Junior Junior Developer 的类，继承自 JuniorDeveloper，并添加初级初级开发人员属性
class JuniorJuniorDeveloper : public JuniorDeveloper {
public:
    bool juniorJuniorDeveloper;
};

// 72. 定义一个名为 Senior Senior Tester 的类，继承自 SeniorTester，并添加高级高级测试人员属性
class SeniorSeniorTester : public SeniorTester {
public:
    bool seniorSeniorTester;
};

// 73. 定义一个名为 Junior Senior Tester 的类，继承自 SeniorTester，并添加初级高级测试人员属性
class JuniorSeniorTester : public SeniorTester {
public:
    bool juniorSeniorTester;
};

// 74. 定义一个名为 Senior Junior Tester 的类，继承自 JuniorTester，并添加高级初级测试人员属性
class SeniorJuniorTester : public JuniorTester {
public:
    bool seniorJuniorTester;
};

// 75. 定义一个名为 Junior Junior Tester 的类，继承自 JuniorTester，并添加初级初级测试人员属性
class JuniorJuniorTester : public JuniorTester {
public:
    bool juniorJuniorTester;
};

// 76. 定义一个名为 Senior Senior QA 的类，继承自 SeniorQA，并添加高级高级质量保证属性
class SeniorSeniorQA : public SeniorQA {
public:
    bool seniorSeniorQA;
};

// 77. 定义一个名为 Junior Senior QA 的类，继承自 SeniorQA，并添加初级高级质量保证属性
class JuniorSeniorQA : public SeniorQA {
public:
    bool juniorSeniorQA;
};

// 78. 定义一个名为 Senior Junior QA 的类，继承自 JuniorQA，并添加高级初级质量保证属性
class SeniorJuniorQA : public JuniorQA {
public:
    bool seniorJuniorQA;
};

// 79. 定义一个名为 Junior Junior QA 的类，继承自 JuniorQA，并添加初级初级质量保证属性
class JuniorJuniorQA : public JuniorQA {
public:
    bool juniorJuniorQA;
};

// 80. 定义一个名为 Senior Senior Product Manager 的类，继承自 SeniorProductManager，并添加高级高级产品经理属性
class SeniorSeniorProductManager : public SeniorProductManager {
public:
    bool seniorSeniorProductManager;
};

// 81. 定义一个名为 Junior Senior Product Manager 的类，继承自 SeniorProductManager，并添加初级高级产品经理属性
class JuniorSeniorProductManager : public SeniorProductManager {
public:
    bool juniorSeniorProductManager;
};

// 82. 定义一个名为 Senior Junior Product Manager 的类，继承自 JuniorProductManager，并添加高级初级产品经理属性
class SeniorJuniorProductManager : public JuniorProductManager {
public:
    bool seniorJuniorProductManager;
};

// 83. 定义一个名为 Junior Junior Product Manager 的类，继承自 JuniorProductManager，并添加初级初级产品经理属性
class JuniorJuniorProductManager : public JuniorProductManager {
public:
    bool juniorJuniorProductManager;
};

// 84. 定义一个名为 Senior Senior Project Manager 的类，继承自 SeniorProjectManager，并添加高级高级项目经理属性
class SeniorSeniorProjectManager : public SeniorProjectManager {
public:
    bool seniorSeniorProjectManager;
};

// 85. 定义一个名为 Junior Senior Project Manager 的类
```


Details:

- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



detailed view



detailed view p2

 Saved: 11/24/2025 9:00:59 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the heroku prod link to this page
- Include pull request link for this feature

URL #1

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=94



URL

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=94



URL #2

[https://dg599-it202-007-](https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=10)

[dev-5af14255b524.herokuapp.com/project/view_car.php?](https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=10)

[id=10](https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=10)

URL #3

[https://github.com/dylangarca/dg599-](https://github.com/dylangarca/dg599-IT202-007/pull/41)

[IT202-007/pull/41](https://github.com/dylangarca/dg599-IT202-007/pull/41)



URL

<https://github.com/dylangarc>



Saved: 11/24/2025 9:01:15 PM

Section #5: (150 pts.) Edit Page

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

```
10 $car_id = (int)$GET["id"];
11
12 if ($car_id <= 0) {
13     flash("Invalid id: " . $car_id, "danger");
14     die(header("Location: list_cars.php"));
15 }
16
17 $db = getDB();
18
19 // fetch existing car data
20 $stmt = $db->prepare("SELECT * FROM cars WHERE id = :id LIMIT 1");
21
22 try {
23     $stmt->execute(["id" => $car_id]);
24     $car = $stmt->fetch(PDO::FETCH_ASSOC);
25
26     if (!$car) {
27         flash("Car not found", "danger");
28         die(header("Location: list_cars.php"));
29     }
30 } catch (Exception $e) {
31     flash("Error fetching car details", "danger");
32     error_log("Error fetching car: " . var_export($e, true));
33     die(header("Location: list_cars.php"));
34 }
```

code that handles loading of the record and the invalid/missing ids

```
1 <!-- HTML Form for editing a car record -->
2 <div class="form-group">
3     <div class="row">
4         <div class="col">
5             <input type="text" value="{$car['name']}"/>
6         </div>
7         <div class="col">
8             <input type="text" value="{$car['year']}"/>
9         </div>
10        <div class="col">
11            <input type="text" value="{$car['make']}"/>
12        </div>
13    </div>
14    <div class="row">
15        <div class="col">
16            <input type="text" value="{$car['model']}"/>
17        </div>
18        <div class="col">
19            <input type="text" value="{$car['price']}"/>
20        </div>
21    </div>
22    <div class="row">
23        <div class="col">
24            <input type="text" value="{$car['description']}"/>
25        </div>
26    </div>
27    <div class="row">
28        <div class="col">
29            <input type="button" value="Save Changes"/>
30        </div>
31        <div class="col">
32            <input type="button" value="Cancel"/>
33        </div>
34    </div>
35 </div>
```

code that generates page

the lists page. After it gets the car data from the database for the specific id and get the car data again if no record is there for the car then it will give the user an error message and redirect them to the list page. For the form generation it is populated with the existing car data and the form will include the proper input types for each field and the proper constraints like required and maxlength. For the submission handling after the successful update the car data will be reloaded and it will reflect the changes. For the styling choices i use bootstrap classes and minimal css styles to keep the same modern and readable appearance. the editable fields are only the fields that the user is able to access when adding their car so that is the make, model, year, type, image_url, and description. The user will not have access to change the id, created, or modified fields.

 Saved: 11/24/2025 10:17:48 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

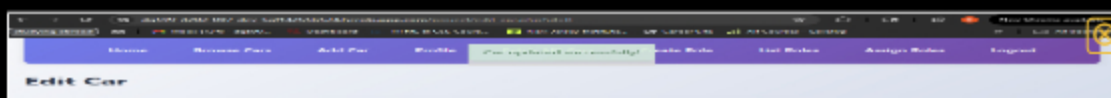
- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



before



after



Make *

Model *

Year *

Type *

Image URL

description

Save

successful update

Make *

Model *

Year *

Type *

Image URL

description

Save

html validation p1

Make *

Model *

Year *

Type *

Image URL

description

Save

html validation p2

Make *

Model *

Year *

Type *

Image URL

description

Save

html validation p3

Make *

Model *

Year *

Type *

Image URL

description

Save

is validation

js validation

The screenshot shows a web browser window with a form titled "Edit Car". The form has several input fields: "Name", "Model", "Year", "Type", "Image URL", and "Description". A JavaScript validation error message is displayed at the top of the form, stating: "Please enter a valid year between 1900 and 2025". The error message is highlighted in a yellow box.

js validation

The screenshot shows a web browser window with a form titled "Edit Car". The form has several input fields: "Name", "Model", "Year", "Type", "Image URL", and "Description". A JavaScript validation error message is displayed at the top of the form, stating: "Please enter a valid year between 1900 and 2025". The error message is highlighted in a yellow box.

php validation



Saved: 11/24/2025 10:32:50 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the heroku prod link to this page
- Include pull request link for this feature

URL #1

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=8



URL

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/view_car.php?id=8



URL #2

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



URL

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



Saved: 11/24/2025 10:33:21 PM

Section #6: (150 pts.) Delete Logic

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```

11 $car_id = (int)$GET["id"] & $filter;
12
13 if ($car_id <= 0) {
14     flash("Invalid car ID", "danger");
15     die(header("Location: list_cars.php"));
16 }
17
18 $ids = getIDs();
19
20 // Fetch the car to check permissions
21 $stmt = $db->prepare("SELECT * FROM Cars WHERE id = :id LIMIT 1");
22
23 try {
24     $stmt->execute(["id" => $car_id]);
25     $car = $stmt->fetch(PDO::FETCH_ASSOC);
26
27     if (!$car) {
28         flash("Car not found", "danger");
29         die(header("Location: list_cars.php"));
30     }
31 } catch (Exception $e) {
32     flash("Error fetching car details", "danger");
33     error_log("Error fetching car for delete: " . var_export($e, true));
34     die(header("Location: list_cars.php"));
35 }

```

delete code that handles deletion

```

36 // Permission check: User can only delete their own cars, OR admin can delete any
37 $can_delete = false;
38 if (has_role("Admin")) {
39     $can_delete = true; // Admin can delete anything
40 } elseif ($car["user_id"] == get_user_id()) {
41     $can_delete = true; // User can delete their own cars
42 }
43
44 if (!$can_delete) {
45     flash("You don't have permission to delete this car", "danger");
46     die(header("Location: list_cars.php"));
47 }
48

```

delete code that handles necessary roles/permission

```

59 // Redirect back to list page, preserving filters if they exist
60 $redirect_url = "list_cars.php";
61 if (!empty($_SERVER['HTTP_REFERER']) && strpos($_SERVER['HTTP_REFERER'], 'list_cars.php') !== false) {
62     // Extract query string from referrer to preserve filters
63     $referer_parts = parse_url($_SERVER['HTTP_REFERER']);
64     if (isset($referer_parts['query'])) {
65         $redirect_url .= "?" . $referer_parts['query'];
66     }
67 }
68
69 die(header("Location: $redirect_url"));
70 }

```

delete code redirect logic

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

For the deletion logic it is getting the car id from the url and then gets the information from the database off of the car id it gets. It will also make sure that the car exists and is a valid car id. Then if the car exists and the user has the correct permission/role the delete command will execute and remove the data of that car from the database. I used a hard delete. It will permanently remove the data from the cars table. Only the authorized user of admin can delete car data. So anyone who doesn't have the role of admin will not be able to delete any car data. After the deletion process occurs the code will redirect the user back to the list page. But if the user was in a filtered/sorted search they will go back to that filtered/sorted search and they won't have to filter search again.



Saved: 11/24/2025 10:45:43 PM

Task #2 (60 pts.) - Examples

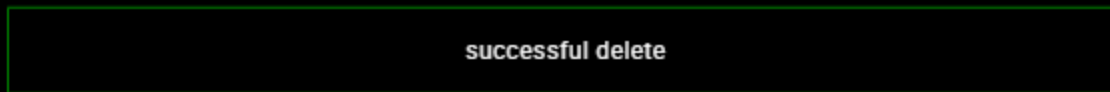
Progress: 100%

Details:

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown

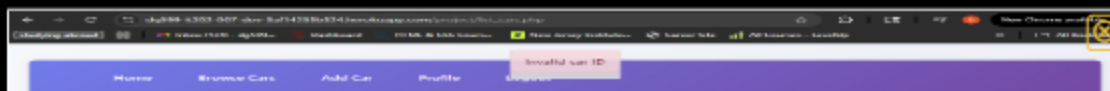
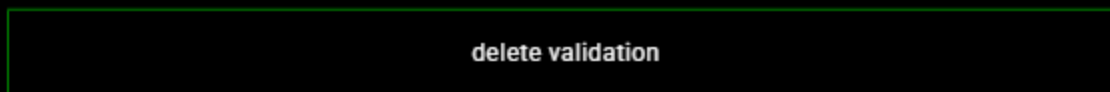


delete.js validation



before delete table

after delete table



delete validation p2



Saved: 11/24/2025 10:55:09 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the heroku prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/delete_car.php



URL

https://dg599-it202-007-dev-5af14255b524.herokuapp.com/project/delete_car.php



URL #2

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



URL

<https://github.com/dylangarca/dg599-IT202-007/pull/41>



Saved: 11/24/2025 10:57:26 PM

Section #7: (50 pts.) Misc

Progress: 100%

Task #1 (16.67 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

Milestone2 #422

1 Merged dylangarca merged 6 commits into `main` from `pull/39/merge` 4 hours ago

Commits (6)

1 Merged dylangarca merged 6 commits into `main` from `pull/39/merge` 4 hours ago

2 Merged dylangarca merged 5 commits into `main` from `pull/40/merge` 4 hours ago

3 Merged dylangarca merged 4 commits into `main` from `pull/41/merge` 4 hours ago

4 Merged dylangarca merged 3 commits into `main` from `pull/42/merge` 4 hours ago

5 Merged dylangarca merged 2 commits into `main` from `pull/43/merge` 4 hours ago

6 Merged dylangarca merged 1 commit into `main` from `pull/44/merge` 4 hours ago



Saved: 11/24/2025 10:59:07 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#)

URL #1

<https://github.com/dylangarca/dg599-IT202/pull/742>

URL

<https://github.com/dylangarca/dg>

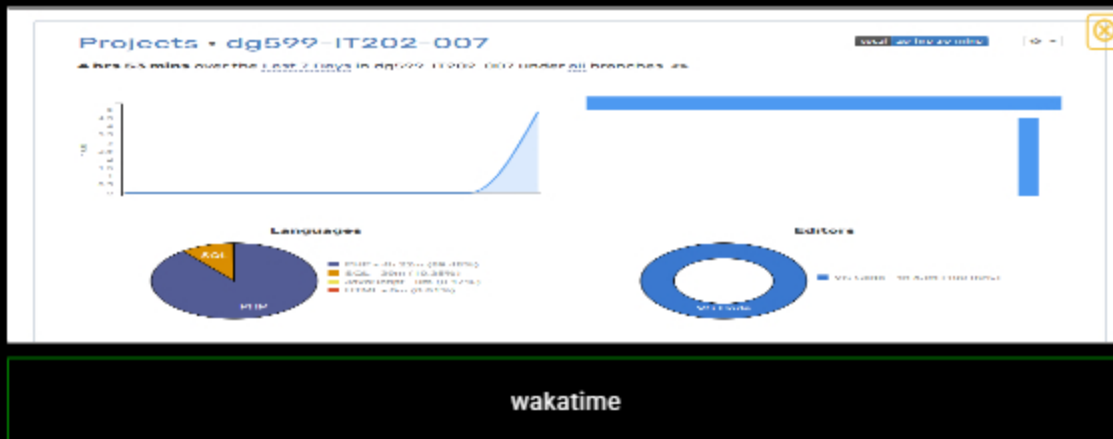
Saved: 11/24/2025 10:59:07 PM

Task #2 (16.67 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary





Saved: 11/24/2025 11:00:30 PM

≡ Task #3 (16.67 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I would say that I learned alot with milestone 2. I think the biggest thing would have to be the api integration with the database. I learned how to desing database tables properly to store api data with the proper columns. I had to think which fields i was going to use from the api and than extract the specific data i was going to use. I also learned about the CRUD operations and how to implement them in webpages.



Saved: 11/24/2025 11:14:33 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I would say that the easiest part of milestone 2 was the create page because it was really similar to the milestone 1 files. It involved many similar things like the 3 layers of validation and the handle submissions and error messages. Since i knew how to do this from milestone 1 that is why it was on the easier side of milestone 2. It was also a straight foward process and it had clear requirements.



Saved: 11/24/2025 11:06:19 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I would say the hardest part of milestone 2 was the api integration with the duplicate handling. The reason for this was i encountered an error with the response structure of the api. So at first i was looking for the data in a data array while it was actually in a results array. Than after I fixed that issue i had to parse the title to get the model and make information. I also encountered a problem with the api becuae until the api integration part i didn't realize that this api does not have the year of the vehicles so i had to first check if the results array had the year of the car and if not set the default to the oldest year.



Saved: 11/24/2025 11:11:44 PM